FeederFlex Phase 2 — System Specification v1.0

A. Executive Summary

FeederFlex Phase 2 freezes the system design so implementation can proceed without re-deciding architecture mid-build. Per the Phase 1 verdict (GO with corrections), this spec assumes: LoRa not WiFi at the household layer, a two-tier BOM (~₹400-500 bare prototype, ~₹800-1,500 field-deployable), constrained heuristic ranking not RL/MILP for the DR controller, linear regression baseline before XGBoost/LightGBM for forecasting, and a synthetic feeder built from real Indian household datasets since no real low-income feeder dataset is public. Nothing here is coded yet — this is the blueprint another engineer (or Claude Code) implements against.

B. Scope

1. What FeederFlex IS: a simulation-first software system that models a synthetic Indian low-income feeder, forecasts near-term load, detects predicted stress, estimates opted-in household flexibility, selects and dispatches curtailment commands under explicit user/safety constraints, and evaluates the result against a no-DR baseline — with a mocked USSD layer, a mocked DISCOM-facing dashboard, and an optional small physical relay/LoRa demo.

2. What FeederFlex IS NOT: not a live DISCOM SCADA integration; not a real telecom USSD deployment; not a validated real-world participation study; not a battery/storage system; not a full commercial VPP platform; not a claim of novel core technology (relay-based DR and feeder aggregation are established prior art per Phase 1 §8).

3. Primary objective: demonstrate, via simulation on real household-derived data, that automation-driven appliance curtailment can reduce a synthetic low-income feeder's evening (5-8pm) peak/shortfall risk, using a non-smartphone household interface.

4. Secondary objectives: produce a DISCOM-compatible-by-design signal schema; produce a reusable simulation/experiment framework (Phase 1 §19, §21 of the research audit); produce an honestly-scoped applied-research write-up.

5. Target users (of the finished demo, not just the eventual product): hackathon judges evaluating technical depth; a hypothetical DISCOM DSM team; a hypothetical low-income household.

6. Target deployment environment (conceptual, not implemented): low-income urban/peri-urban Indian residential feeders, 50-200 households.

7. Target household characteristics: no assumed smartphone, no assumed home WiFi/broadband, feature-phone reachable via USSD/missed-call/SMS. Households may have zero or more eligible flexible appliances — participation and appliance eligibility are explicitly modeled as variables (Part W Experiment E sweeps flexible-appliance prevalence at 40/60/80/100%), not assumed universal. **Household income is not directly modeled** unless a selected Phase 3 dataset specifically supports it — the synthetic feeder represents a low-income/peri-urban deployment *scenario*, built from real Indian household consumption profiles, not a dataset labeled by income (revised per external review, see Part AK).

8. Target feeder concept: a synthetic **11 kV distribution feeder** serving a modeled set of downstream residential connections through distribution transformers (DTs) — matching how Indian DISCOMs actually structure LT distribution and how RDSS/NFMS instruments feeders (Phase 1 §15). Simplified for this project as:

```
11 kV Feeder F-001
       |
       +-- DT-01 -> Households 1-50
       +-- DT-02 -> Households 51-100
       +-- DT-03 -> Households 101-150
```

No power-flow analysis is performed — this models aggregate active-power demand at feeder and DT level only, not voltage/impedance. **[VERIFY IN PHASE 3]** the specific claim that NFMS monitors at 11 kV was raised in external review and is consistent with Phase 1's RDSS feeder-meter findings but wasn't independently re-confirmed in this revision — treat the 11 kV framing as a reasonable, standard-practice engineering assumption (Part AD), not a re-verified fact.

9. Inside one household: an appliance controller (conceptually a relay+ESP32+LoRa node, simulated in software for this phase) that reports appliance state and executes curtail/restore commands within user-set permissions.

10. At feeder level: aggregation of household states into a feeder load signal, and dispatch of DR commands to the selected subset of opted-in appliances.

11. At the central/backend level: data ingestion, synthetic feeder construction, forecasting, stress detection, flexibility estimation, the DR controller, simulation orchestration, and the database of record.

12. What the DISCOM/operator sees: a mocked dashboard showing feeder ID, current/forecast load, available flexibility (as a range, not a fixed number, per Phase 1 §7/§20), active DR events, and reduction achieved — schema designed for IEC 60870-5-104 compatibility, **not** live-integrated.

13. What the household sees (simulated): a USSD/missed-call menu for opt-in, appliance preference setting, DR event notice, and incentive status.

14. What the algorithm controls: only appliances the household has explicitly marked flexible/interruptible, only within the household's own stated limits.

15. What remains under user control: critical-appliance designation, per-appliance permission level, override/opt-out at any time, emergency stop.

16. What is simulated: the feeder itself, household participation, DR event outcomes, LoRa communication, USSD sessions, DISCOM integration.

17. What is real: the household/appliance-level historical data used to build the synthetic feeder (iAWE, Kaggle Indian smart-meter datasets, Batra et al. ownership data — subject to Phase 3 re-verification per RULE 3); weather/solar data (Open-Meteo, confirmed live/free in Phase 1 §28); national/state grid demand context (Grid-India, confirmed live in Phase 1 §28).

18. What is optional: the physical ESP32/relay/LoRa demo nodes (Level 2/3 only, see Part AG); a live telecom USSD sandbox (explicitly out of scope, see Part AF).

Scope boundary rule: if a feature requires data, access, or infrastructure not confirmed available in Phase 1/Phase 3, it is designed and simulated, never claimed as live.

C. Actors and Roles

Actor	Responsibilities	Inputs	Outputs	Can control	Cannot control
Household/User	Register, set appliance permissions, opt in/out, override	USSD menu selections	Preferences, consent state, override commands	Own appliance permissions, own participation, emergency override	Other households, feeder-level decisions, forecast/algorithm logic
Appliance	Report state, execute commands within permission	Commands from Household Controller	Status (on/off/power draw)	Nothing (passive, obeys controller within permission bounds)	Its own eligibility (set by user)

Actor	Responsibilities	Inputs	Outputs	Can control	Cannot control
Household Controller	Local decision point: hold user permissions, execute/refuse DR commands, enforce fail-open safety	Backend commands, user preferences, local appliance state	Status reports, command acks	Fail-open default, local override honoring	Cannot act outside user-set permissions, cannot override critical-load protection
LoRa Gateway (simulated)	Aggregate household messages, relay to backend, relay commands down	Household status messages	Aggregated feeder telemetry	Nothing decision-wise (relay only)	Cannot originate commands
FeederFlex Backend	Orchestrate forecasting, stress detection, flexibility estimation, DR controller, logging	Feeder telemetry, forecasts, user preferences	DR commands, dashboard data	Dispatches within constraints	Cannot violate RULE 11 (never select non-consented appliance)
Forecasting Engine	Predict near-term feeder load	Historical + weather features	Load forecast + confidence	Model selection, retraining	Cannot dispatch commands directly
Flexibility Engine	Estimate available flexible kW per household/feeder	Appliance states + permissions + constraints	Flexibility estimate (min/expected/max)	Nothing dispatch-related	Cannot override user permissions
DR Controller	Rank and select appliances, dispatch, monitor, restore	Required reduction + flexibility estimate + constraints	Dispatch commands, restore commands	Selection within eligible pool only	**Cannot select non-consented, critical, cooling-down, or over-limit appliances (RULE 11)**
Feeder Simulator	Run the synthetic feeder, apply DR effects, produce baseline vs. controlled comparison	Household profiles, DR actions	Simulated load curves, metrics	Simulation parameters (Phase 1 §19/§21)	Not a live system
DISCOM/Operator	View feeder status, review DR events (read-only in this phase)	Dashboard access	None (Phase 2 scope is read-only for this actor)	Nothing — no live control authority granted in this phase	**Cannot force an appliance directly** (no live path exists in Phase 2 scope)
Administrator	Configure system parameters, manage households/feeders in the simulation, view audit logs	Admin console access	Configuration changes	Simulation/system config	Cannot bypass household-level consent

Decision-authority answers (explicit, per the brief's requirement):

- **WHO decides which appliances are eligible?** The household user, at registration/preference time (RULE 9). The system never auto-assigns eligibility.
- **WHO decides when a DR event happens?** The Backend, triggered automatically by the Stress Detection logic (Part L) — not a human operator in this phase, since no live DISCOM control path exists.
- **WHO decides which eligible appliances are activated?** The DR Controller's ranking algorithm (Part N), constrained to the eligible pool only.
- **WHO can override a DR action?** The household user, at any time, via the Household Controller's local override (fail-open default) or the USSD override flow (Part G/H).
- **WHO can opt out?** The household user — permanently (unregister), temporarily (pause), or per-event (decline/cancel).

D. Functional Requirements

ID	Requirement
FR-001	System shall construct a synthetic feeder from real Indian household-level datasets, never presenting it as real feeder data (RULE 5)
FR-002	System shall forecast feeder-level load at a defined horizon and resolution (Part K)
FR-003	System shall detect predicted feeder stress against an explainable, documented threshold (Part L)
FR-004	System shall estimate available household and feeder flexibility as theoretical, currently-available, and usable-after-constraints values (Part M)
FR-005	System shall never select a non-consented, critical, cooling-down, or interruption-limit-exceeded appliance for curtailment (RULE 11)
FR-006	System shall allow per-appliance, per-household user permission configuration, including a non-binary AC control model (not auto-controllable)

ID	Requirement
FR-007	System shall support user override/opt-out at registration, preference, and active-event stages (RULE 12)
FR-008	System shall stagger appliance restoration to reduce rebound peak (Part P)
FR-009	System shall log every DR action, override, and event outcome for evaluation and audit
FR-010	System shall present a DISCOM-facing dashboard schema designed for IEC 60870-5-104 compatibility, never claiming live integration (RULE 4)
FR-011	System shall simulate a USSD/missed-call household interface, clearly labeled as simulated (Part H)
FR-012	System shall run configurable experiments varying household count, participation rate, and shortfall severity (Part W)
FR-013	System shall compute the evaluation metrics defined in Part X and compare against the no-DR baseline
FR-014	System shall fail open (restore normal appliance operation) on any communication, gateway, or backend failure (Part Q)

E. Non-Functional Requirements

ID	Category	Requirement (engineering target, not a measured result)
NFR-001	Reliability	Simulation runs shall complete deterministically given a fixed random seed

ID	Category	Requirement (engineering target, not a measured result)
NFR-002	Safety	No code path shall leave a simulated appliance in a state the user did not permit; fail-open is the default on any failure branch
NFR-003	Explainability	The DR ranking score (Part N) shall be a transparent weighted formula, not a black-box model, so any selection can be explained per-appliance
NFR-004	Latency	A single simulated DR decision cycle (forecast to dispatch) should target under 2 seconds on a laptop-class machine for a 200-household feeder — engineering target only
NFR-005	Scalability	The architecture should not hard-code assumptions that break above 200 households, but is not required to be load-tested beyond that in this phase (Phase 1 §20/§21 named this untested)
NFR-006	Maintainability	Forecasting, flexibility estimation, and the DR controller shall be separable modules with defined interfaces, not one monolithic script
NFR-007	Observability	Every DR event, override, and restoration shall be logged with enough detail to reconstruct the decision (audit_logs table, Part S)
NFR-008	Privacy	Simulated household identifiers shall not require or store real PII; synthetic households only
NFR-009	Fault tolerance	Backend failure mid-event shall not require manual appliance recovery — the fail-open default handles this locally

F. Household Appliance Model

Categories (using correct terminology, not "useless appliances"):

- **Critical loads:** refrigerator (default), medical equipment (default), essential lighting (default), any appliance the user explicitly marks critical — never eligible for DR regardless of algorithm output.
- **Flexible/shiftable candidates:** water pump, geyser/water heater, washing machine, select cooling loads (coolers, fans), EV charging if present — eligible only if the user opts them in.
- **AC — explicitly not auto-controllable.** Per RULE 8, AC defaults to "Never control" and requires explicit user action to change:

```
AC control level (user-set, default = Never control):
[ ] Never control
[ ] Allow limited DR (curtail up to X minutes, user sets X)
[ ] Allow only above/below a comfort condition (if temperature sensing is
available — FUTURE WORK, not in Phase 2 scope)
```

The same three-tier permission pattern applies to every appliance category, not just AC. **MVP control model, revised per external review:** the implemented algorithm performs **temporary curtailment and delayed (staggered) restoration of currently-running opted-in flexible loads only** — it does not implement general load-shifting. `shiftable` remains a field on the appliance model to record which appliances *could* support deferred-start scheduling (e.g. washing machines, pumps started later rather than curtailed mid-run), but deferred-start scheduling itself is explicit **FUTURE WORK**, not part of the Part N MVP algorithm. Wording elsewhere in this spec has been corrected to "curtailment and delayed restoration," not "shift or curtail," to avoid promising a feature the algorithm doesn't implement.

Appliance data model:

Field	Type	Source
appliance_id	UUID	C. System-generated
household_id	UUID (FK)	C. System-generated
appliance_type	enum (pump, geyser, cooler, AC, fridge, washing_machine, other)	B. User configuration at registration
rated_power_w	float	A. Dataset (where available) or B. user-entered estimate
current_power_w	float	D. Simulation (derived from dataset load curve)

Field	Type	Source
criticality	enum (critical, flexible)	B. User configuration, defaulting per category
interruptible	boolean	B. User configuration
shiftable	boolean	B. User configuration
user_permission	enum (never, limited, conditional)	B. User configuration
minimum_runtime_minutes	int	C. System assumption per appliance_type, overridable by user
cooldown_minutes	int	C. System assumption per appliance_type (Part O ties this to fairness)
minimum_off_time_minutes	int	C. System assumption
preferred_operating_window	time range, nullable	B. User configuration
priority	int (1=highest protect, 5=most flexible)	C. System-derived from criticality + user_permission
current_state	enum (on, off, curtailed, cooling_down)	D. Simulation
last_interruption_time	timestamp, nullable	D. Simulation
cumulative_interruptions	int	D. Simulation (feeds fairness, Part O)
recovery_requirement	enum (immediate, staggered)	C. System assumption per appliance_type
current_run_started_at	timestamp, nullable	D. Simulation — added per external review, makes minimum_runtime a computable condition
runtime_since_start_minutes	computed (now - current_run_started_at)	D. Simulation — minimum_runtime_satisfied = runtime_since_start_minutes >= minimum_runtime_minutes

Field	Type	Source
max_interruption_minutes_per_event	int	B. User configuration — replaces the old single max_interruption_minutes field (was ambiguous between duration and count, fixed per external review)
daily_interruption_minutes_limit	int	B. User configuration, defaulting per appliance_type — cumulative daily cap, distinct from per-event cap
cumulative_interruption_minutes_today	float	D. Simulation — resets daily; checked against daily_interruption_minutes_limit

Source legend applied consistently: A = from dataset, B = from user configuration, C = system/engineering assumption (to be documented in the Assumption Register, Part AD), D = generated during simulation.

G. User Control Model

Flow, in order: (1) Registration — phone number, household ID created. (2) Household setup — approximate location/feeder assignment (simulated). (3) Appliance registration — user lists appliances via USSD menu. (4) Appliance flexibility selection — per-appliance permission (never/limited/conditional, Part F). (5) Preference configuration — max interruption, preferred windows. (6) Opt-in — explicit consent step, separate from registration (a household can register without opting into DR). (7) DR event notification — simulated USSD/SMS push when an event affects the household. (8) Accept/decline — for non-automated events only; automated events proceed unless overridden (matches POWBAL precedent from Phase 1 §3, where automation outperformed opt-in-per-event). (9) Temporary opt-out — pause for N hours/days without full unregistration. (10) Emergency override — immediate, always available, no confirmation friction. (11) Incentive/status display — accumulated credit, via USSD query. (12) History — past events and interruptions, via USSD query.

Explicit answers:

- **Can the operator force an appliance?** No. No actor in this architecture has a path to bypass household consent (RULE 11, RULE 9). The DISCOM/operator role is read-only in Phase 2 (Part C).

- **Can the operator override user preferences?** No.
- **Can the user immediately cancel participation? Contradiction fixed per external review** — see the explicit action/effect table below; emergency override and active-event opt-out are both immediate, only a preference change (not tied to an active event) and permanent unregistration are next-cycle/future-event.
- **What happens when communication fails?** Fail-open — the Household Controller defaults to normal (uncurtailed) appliance operation (Part Q).
- **What happens when a user changes preferences during an active event?** The change is accepted immediately for future dispatch decisions; an already-issued curtailment for the current event either respects the user's new override (if they overrode) or completes its planned duration (if unchanged) — never extended beyond what was already dispatched.
- **What happens if an appliance is currently unavailable (e.g., already off)?** It contributes zero to available flexibility for that cycle and is excluded from ranking (Part M/N).

Action	Effect
Emergency override	Immediate — cancels the active curtailment on that appliance right now
Active-event opt-out (this event only)	Immediate — same effect as override for the current event
Preference change (no active event, or not targeting the active event)	Applies to the next DR decision cycle
Permanent unregister / pause	Applies to future events only; does not retroactively affect an already-dispatched curtailment

Permission hierarchy (highest to lowest authority): Household user (own appliances) > Household Controller's local fail-open safety logic > DR Controller's dispatch decision > Backend's stress-triggered event initiation. No actor above the household user can compel an appliance action against that household's stated permission.

H. USSD / Missed-Call Model (SIMULATED unless a real telecom integration is added later)

Menu (revised from the brief's example, with a preferences submenu and a clearer top-level structure):

```
*123# FeederFlex
1. Join / My Status
2. My Appliances
3. DR Preferences
4. Current DR Event
5. Incentives
6. Opt Out / Pause
0. Help
```

Submenu — 2. My Appliances: list registered appliances → select one → set type (pump/geyser/cooler/AC/other) → set permission level (never/limited/conditional) → confirm.

Submenu — 3. DR Preferences: set max interruption minutes → set preferred no-disturb window → save.

Submenu — 4. Current DR Event: shows active event status ("Cooler paused, resumes in 12 min") and offers an immediate override option ("Press 9 to resume now").

Submenu — 6. Opt Out / Pause: pause for 24h / pause for 7 days / unregister — each with a one-line confirmation, no multi-step friction (matches the missed-call/USSD banking pattern already familiar to this demographic, per Phase 1 §11).

Backend interaction: each USSD keypress maps to a simulated session state machine → an API call (Part T) → a database write (Part S: ussd_sessions, household_preferences) → a confirmation string returned to the (simulated) session.

Confirmation messages: every write action (registration, permission change, opt-out) returns an explicit one-line confirmation string in the simulated session log — this is what stands in for a real USSD confirmation screen.

DR notification flow (simulated SMS/missed-call, not USSD-session-based since it's push not pull): event start → simulated outbound message logged → household's stated override window applies.

This entire section is a **simulated interaction layer** — no telecom aggregator, no real USSD gateway, no real SIM/phone number handling in Phase 2. Real integration is explicitly FUTURE WORK (Part AF).

I. System Architecture

A. Text architecture diagram

```
DATA SOURCES
  - Real: iAWE / Kaggle Indian smart-meter datasets / Batra et al. ownership
data (Dataset A/B/C, Part AC)
  - Real: Open-Meteo (weather/solar radiation, live -- Dataset D, Part AC)
  - Real (contextual): Grid-India (national/state demand + generation mix,
live)
    |
    v
DATA INGESTION
    |
    v
DATA PROCESSING
    |
    v
SYNTHETIC FEEDER  (11 kV feeder, DT-structured, Part B item 8)  [SIMULATED]
    |
    +----------------------------+
    v                            v
LOAD FORECAST                RENEWABLE FORECAST  (Part K2)
(demand, Part K)              [SIMULATED conversion from real radiation]
    |                            |
    +------------+---------------+
                 v
        NET LOAD / SHORTFALL  (forecast_demand - forecast_renewable, Part L)
                 |
                 v
        FEEDER STRESS DETECTION  (renewable-aware trigger, Part L)
                 |
         DR event required?
           /          \
         NO            YES
          |              v
          |      FLEXIBILITY ESTIMATION ENGINE (Part M)
```

```
          |              |
          |              v
          |      DR CONTROLLER  (Part N -- curtailment + delayed restoration)
          |              |
          |              v
          |      APPLIANCE COMMANDS -> HOUSEHOLDS
          |              |
          |              v
          |      STAGGERED RESTORE  (Part P)
          |              |
          +------+-------+
                 v
        FEEDER LOAD UPDATE
                 |
                 v
            EVALUATION  (baseline vs controlled, Part X)

PARALLEL LAYERS:
USER / USSD LAYER  <-> BACKEND  (Part G/H)
BACKEND  <-> DATABASE  (Part S)
BACKEND  <-> DISCOM DASHBOARD  (read-only mock, Part T/Y)
BACKEND  <-> OPTIONAL LORA HARDWARE  (Level 2/3 only)
```

B. Mermaid diagram

```
flowchart TD
    DS1[Real: iAWE / Kaggle household datasets] --> ING[Data Ingestion]
    DS2[Real: Open-Meteo weather/solar radiation] --> ING
    DS3[Real: Grid-India demand context] --> ING
    ING --> PROC[Data Processing]
    PROC --> FEEDER[Synthetic 11kV Feeder Builder - DT structured]
    FEEDER --> FCST[Load Forecast - Part K]
    FEEDER --> PVCONV[Synthetic PV Conversion Model - Part K2]
    PVCONV --> RFCST[Renewable Forecast - Part K2]
    FCST --> NETLOAD[Net Load / Shortfall Calc]
    RFCST --> NETLOAD
    NETLOAD --> STRESS[Renewable-Aware Stress Detection - Part L]
    STRESS -->|DR needed| FLEX[Flexibility Estimation Engine - Part M]
    STRESS -->|no DR needed| EVAL[Evaluation / Metrics - Part X]
    FLEX --> DR[DR Controller - Part N]
    DR --> CMD[Appliance Commands: Curtail]
    CMD --> HH[Household Controllers]
```

```
    HH --> RESTORE[Staggered Restore - Part P]
    RESTORE --> FEEDER2[Feeder Load Update]
    FEEDER2 --> EVAL
    FEEDER2 --> FCST

    USER[Household User] <--> USSD[USSD / Missed-Call Layer - simulated]
    USSD <--> BACKEND[FeederFlex Backend]
    BACKEND <--> DB[(Database)]
    BACKEND <--> DASH[DISCOM Dashboard - read-only mock]
    BACKEND <--> LORA[Optional Physical LoRa Hardware - Level 2/3]
    BACKEND --> FCST
    BACKEND --> STRESS
    BACKEND --> FLEX
    BACKEND --> DR
    HH --> BACKEND
```

Layer summary, revised per external review: the pipeline now has two forecast branches (demand and renewable) that merge into a net-load/shortfall calculation before stress detection — this is the structural fix for the "drifted from renewable intermittency" finding, and is the single most important change in this revision. The horizontal layers (USSD, Backend, Database, Dashboard) are unchanged. The optional LoRa hardware, if built, still only replaces the simulated household-to-gateway link and does not change anything above the Household Controller layer.

J. Detailed Data Flow — One Complete DR Event

Step	Transition
T0	Feeder simulator holds current aggregate load (sum of household profiles at this timestep)
T1	Forecasting Engine predicts load N steps ahead (Part K sets the exact horizon)
T1b	Renewable Forecasting Engine (Part K2) predicts renewable availability over the same horizon; net_load = forecast_demand - forecast_renewable is computed. Added per external review -- this is the step that was previously missing and is what actually connects the project to the renewable-intermittency challenge.

Step	Transition
T2	System reads the feeder's configured available_supply_kw (renewable + firm grid import) and safe_threshold_kw (Part L, revised)
T3	Stress Detection compares net_load vs. safe_threshold_kw; if net_load exceeds the threshold minus safety margin, state moves from Normal to Warning/Critical (revised to be renewable-aware, not demand-only)
T4	Flexibility Engine queries all eligible households (opted-in, not currently in cooldown) on this feeder
T5	Flexibility Engine computes theoretical, currently-available, and usable-after-constraints flexibility per household, then aggregates to feeder level (Part M)
T6	DR Controller computes required_reduction = net_load_kw - safe_threshold_kw
T7	DR Controller ranks all eligible appliances by the scoring formula (Part N)
T8	DR Controller selects the minimum-necessary subset meeting required_reduction, filtering out any appliance failing a hard constraint (non-consent, critical, cooldown, over-limit — RULE 11)
T9	Commands are issued to selected Household Controllers (simulated message, Part R)
T10	Household Controllers apply the command; appliance current_state moves to "curtailed"; last_interruption_time and cumulative_interruptions update
T11	Feeder Simulator recalculates aggregate load with the curtailment applied

Step	Transition
T12	DR Controller monitors the event for its planned duration, watching for user overrides (Part G) which remove an appliance from the active event immediately
T13	At planned duration end, restore commands are issued
T14	Restoration is staggered per a restart schedule (Part P), not simultaneous, to limit rebound peak
T15	Results (pre/post load, appliances controlled, overrides, duration, rebound peak) are logged to dr_events and dr_actions (Part S) for evaluation (Part X)

Each step is a discrete function call in the eventual implementation, not a continuous process — this keeps the pipeline testable stage-by-stage, which matters for a hackathon build timeline.

K. Load Forecasting Design

Target variable: synthetic feeder aggregate active power (kW) at each 15-minute interval.

Prediction horizon: 1 to 4 steps ahead (15-60 minutes) — matched to the 5-8pm evening window's decision-relevant lead time (Phase 1 §3, §12), not a long-horizon forecast.

Time resolution: 15 minutes, matching the Phase 1 §16 recommendation for the initial synthetic feeder.

Input features:

- Historical load: lagged feeder load (t-1, t-2, t-4, t-96 i.e. same time yesterday)
- Calendar: hour of day, day of week, weekend flag
- Rolling statistics: rolling mean/std over the last 2 hours
- Weather: temperature, cloud cover, shortwave radiation from Open-Meteo (confirmed live-available, Phase 1 §28) — radiation as a proxy for solar-driven net-load shape, temperature as a proxy for cooling-load demand
- Humidity: included if available from the same weather source, justified as a cooler/fan-usage proxy

Models:

- **Baseline:** Linear Regression — required first per Phase 1 §10/RULE 6, gives an explainable floor to beat
- **Candidate:** XGBoost or LightGBM — the recommended primary model (Phase 1 §12)
- **Not used by default:** LSTM/Transformer — no evidence in Phase 1's research that the data volume/complexity justifies this; would only be reconsidered if the tree-based model demonstrably underfits a clear temporal pattern the simpler models can't capture, which should be shown empirically before adding complexity

Train/validation/test split: chronological split (not random shuffle, to prevent leakage) — e.g. first 70% of the synthetic time series for training, next 15% for validation/hyperparameter tuning, final 15% held out as test. No test-period data may appear in any lag/rolling feature computation for training rows (leakage prevention).

Feature engineering: all lag and rolling features computed strictly from data available at or before the prediction time (no look-ahead).

Evaluation metrics: MAE and RMSE against the held-out test set (Part X has exact formulas).

Retraining strategy: for Phase 2/hackathon scope, a single train-once model is sufficient — no live retraining loop is required; note this as a FUTURE WORK item for a real deployment.

Model storage: serialized via joblib, loaded by the backend at startup.

Inference process: at each simulation timestep, the backend calls the forecasting module with the current feature window, receives a point forecast **and a simple empirical prediction interval computed from validation-set residuals** (e.g. forecast ± the residual standard deviation, or a residual-quantile band), and passes both directly into Stress Detection (Part L / T1-T3 in Part J). **Confidence-interval inconsistency fixed per external review:** this was previously described as optional in this section but implied as required by the dashboard (Part U/Y). It is now a required MVP output, using the simplest defensible method (empirical residual bounds), not a novel ML component — this also gives Part Y's "flexibility shown as a range, never a guaranteed number" pattern a matching "forecast shown as a range" counterpart on the demand side.

Data depth requirement, added per external review: because lag features include t-96 (same time yesterday) and rolling multi-hour statistics, the training dataset must span **multiple consecutive days to weeks**, not a single 24-hour example — a one-day dataset

cannot support these features. Phase 3 must confirm the actual number of consecutive days available in the selected household datasets (Part AC) before the exact lag/rolling feature set is finalized.

K2. Renewable Availability Forecasting (NEW, added per external review)

Why this exists: the original Part K forecast demand only. The hackathon challenge is specifically about renewable-supply shortfall (Phase 1 §2), not generic peak-load management, so a renewable-supply-side forecast is required, not optional.

Important scope clarification: FeederFlex does **not** assume household rooftop solar. This is an **upstream/local renewable availability scenario** — representing whatever mix of solar (and optionally other renewables) is assumed to feed this feeder from the grid side, not panels on the households being modeled. Households still only ever contribute demand flexibility (Part M).

Data chain (labeled explicitly, since solar radiation is not generation):

```
Open-Meteo solar radiation (REAL, live/historical, Phase 1 s28)
        |
        v
Synthetic PV availability model (SIMULATED conversion step)
  -- a simple, documented function: pv_output_kw = capacity_kw *
performance_ratio * (ghi / 1000)
     with capacity_kw a scenario parameter (Part AD), performance_ratio a
standard
     PV-derating constant (~0.75-0.85, cited engineering range, not India-
specific-sourced)
        |
        v
Renewable generation profile for the feeder scenario (SIMULATED, labeled as
such in Part Y)
```

Alternative if Phase 3 finds better data: if a real regional renewable-generation-by-fuel-type series is confirmed accessible (Phase 1 s28 noted Grid-India publishes generation mix at state/regional level), that can substitute for or validate the synthetic PV model's shape — **VERIFY IN PHASE 3**, not assumed available at feeder scenario granularity.

Forecast target: forecast_renewable_kw, same horizon and resolution as the demand forecast (Part K), so both feed the same Part L trigger cycle.

Model: the same model-selection logic as Part K applies — start with a simple regression from radiation forecast to PV output (near-deterministic given the conversion formula above, so this is closer to a calculation than an ML problem), no case for XGBoost/LSTM here since the physical relationship is simple and well-defined.

L. Shortfall / Feeder-Stress Detection

States: Normal, Warning, Critical, DR Event, Insufficient Flexibility, Recovery. **Insufficient Flexibility added as an explicit state per external review** (previously only logged as an event flag in Part N, not represented in the state machine itself).

Trigger condition — revised to be renewable-aware per external review. The original version compared forecast demand against feeder capacity alone, which is a peak-load-management trigger, not a renewable-intermittency trigger — it didn't actually connect to the hackathon challenge's core ask (Phase 1 §2: keep electricity dependable when *renewable supply* falls below demand). The corrected trigger uses net load:

```
# forecast_renewable_kw comes from Part K's renewable/PV forecast sub-model
forecast_net_load_kw = forecast_demand_kw - forecast_renewable_kw

# firm_grid_import_kw = the portion of upstream supply treated as reliably
available
# (a C. system assumption / scenario parameter, Part AD -- this is what lets
the
# project avoid claiming any specific real DISCOM's actual import capacity)
available_supply_kw = forecast_renewable_kw + firm_grid_import_kw

safe_threshold_kw = available_supply_kw * (1 - safety_margin)

if forecast_demand_kw <= safe_threshold_kw * 0.90:
    state = NORMAL
elif safe_threshold_kw * 0.90 < forecast_demand_kw <= safe_threshold_kw:
    state = WARNING
elif forecast_demand_kw > safe_threshold_kw and forecast_demand_kw <=
available_supply_kw:
    state = CRITICAL          # trigger DR event
elif forecast_demand_kw > available_supply_kw:
    state = CRITICAL          # trigger DR event at maximum available
flexibility

# after DR Controller (Part N) runs:
```

```
IF achieved_reduction_kw < required_reduction_kw:
    state = INSUFFICIENT_FLEXIBILITY   # explicit state, not just a logged
flag

# after a DR event's planned duration elapses and appliances are restoring:
state = RECOVERY  # until forecast_demand_kw stabilizes below
safe_threshold_kw * 0.90, then NORMAL
```

`firm_grid_import_kw` and `safety_margin` are scenario parameters, not measured values from a real DISCOM (Part AD). `forecast_renewable_kw` comes from Part K's renewable forecast sub-model, itself derived from Open-Meteo solar radiation via a synthetic PV conversion model (Part K), not from real local generation data, since none exists for a hypothetical feeder (Part Y).

This remains a simple threshold-band rule, not a probabilistic/ML-based trigger — auditable for a judge or DISCOM reviewer to inspect directly, and now demonstrably tied to renewable shortfall rather than generic peak-load management, closing the gap flagged in external review as the single most important conceptual correction to this spec.

M. Flexibility Estimation Engine

Three-tier flexibility, per appliance, then aggregated:

- **Theoretical flexibility** = rated_power_w if the appliance category is ever flexible (ignores current state/permission — an upper bound only).
- **Currently-available flexibility** = theoretical, filtered to appliances that are (a) currently on/drawing power, and (b) user_permission != never.
- **Usable flexibility after constraints** = currently-available, further filtered by: not in cooldown, max_interruption_minutes not already exhausted this window, minimum_runtime satisfied, not in preferred_operating_window (unless user_permission = conditional and the condition is met).

Aggregation:

```
household_flexibility_kw = sum(usable_flexibility_i for appliance i in
household)
feeder_flexibility_kw = sum(household_flexibility_h for household h in
feeder, h.opted_in == true)
```

Data structure (per feeder, per timestep):

```
{
  "feeder_id": "F-001",
  "timestamp": "2026-10-15T18:15:00+05:30",
  "theoretical_flexibility_kw": 42.0,
  "currently_available_kw": 18.5,
  "usable_after_constraints_kw": 11.2,
  "household_breakdown": [ {"household_id": "H-014", "usable_kw": 0.8,
"eligible_appliance_ids": ["A-051"]} ]
}
```

The DR Controller (Part N) only ever acts on `usable_after_constraints_kw` — the other two tiers exist for reporting/dashboard context (Part T/Y) and should never be presented to a DISCOM as "available flexibility" on their own (this is exactly the "don't guarantee flexibility" lesson from Phase 1 §7/§21 Q15).

N. Demand Response / Optimization Algorithm

Approach: constrained heuristic/ranking, not RL/MILP (Phase 1 §11, RULE 14).

Scoring formula — revised per external review (criticality dropped from scoring since it's already a hard filter, not a weighted term; urgency defined explicitly; an explicit minimum-intervention objective added):

The controller shall select the **minimum number of appliances and minimum total interruption duration** needed to meet the target reduction, subject to all hard constraints below — this is the controller's explicit objective, not just an incidental property.

```
score_i = (w1 * fairness_penalty_i)      # Part O — recently-interrupted
appliances penalized upward
        + (w2 * rebound_risk_i)           # Part P — high simultaneous-
restart-risk appliances penalized upward
        + (w3 * interruption_cost_i)       # household-stated
priority/inconvenience, 1 (low cost) to 5 (high cost)
        - (w4 * flexibility_kw_i)          # bigger flexible load preferred,
reduces the number of appliances needed

urgency_i = time_since_appliance_last_eligible_and_unused_minutes
# used only as a tie-breaker among equal-score candidates, not a weighted
score term:
# among ties, prefer the appliance with the LOWER urgency (i.e. one that
hasn't
# recently "missed its turn") so no single appliance is skipped indefinitely
```

Weights (w1-w4) are C. system assumptions/tunable parameters (Part AD), not claimed as optimized values. Criticality and consent are enforced entirely in the FILTER step below, never as a scoring term — an appliance either passes the hard constraints or is never scored at all.

Weights (w1-w5) are C. system assumptions/tunable parameters, to be documented in Part AD and adjusted empirically during Phase 3/implementation — not claimed as optimized values.

Pseudocode:

```
FUNCTION run_dr_cycle(feeder, required_reduction_kw):
    # INPUT
    candidates = get_all_appliances(feeder)

    # FILTER  (hard constraints -- RULE 11, never violated)
    # BUG FIXED per external review: was comparing a duration field against a
    # count-shaped limit name. Now uses the corrected Part F fields
consistently.
    eligible = [a for a in candidates if
                a.user_permission != NEVER
                and a.criticality != CRITICAL
                and a.current_state == ON
                and a.cooldown_remaining == 0
                and a.runtime_since_start_minutes >=
a.minimum_runtime_minutes
                and current_event_duration_estimate <=
```

```
a.max_interruption_minutes_per_event
                and (a.cumulative_interruption_minutes_today +
current_event_duration_estimate) <= a.daily_interruption_minutes_limit]

    # RANK
    for a in eligible:
        a.score = compute_score(a)   # Part N scoring formula, criticality
excluded (already filtered above)
    ranked = sort(eligible, by=score, ascending=True, tiebreak=urgency_i
ascending)

    # SELECT  -- minimum-necessary-intervention objective (explicit per
external review)
    selected = []
    cumulative_kw = 0
    for a in ranked:
        if cumulative_kw >= required_reduction_kw:
            break
        selected.append(a)
        cumulative_kw += a.flexibility_kw

    # VERIFY CONSTRAINTS  (re-check immediately before dispatch -- state may
have changed)
    selected = [a for a in selected if still_eligible(a)]

    # DISPATCH
    for a in selected:
        send_curtail_command(a)
        a.current_state = CURTAILED
        a.last_interruption_time = now()
        a.cumulative_interruptions += 1
        a.cumulative_interruption_minutes_today +=
current_event_duration_estimate

    # MONITOR  (Part J, T12)
    schedule_event_monitor(selected, planned_duration)

    # RESTORE  (Part P handles staggering)
    schedule_staggered_restore(selected)

    # LOG
    log_dr_event(feeder, selected, required_reduction_kw, cumulative_kw)
```

```
    IF cumulative_kw < required_reduction_kw:
        log_insufficient_flexibility_event(feeder, shortfall =
required_reduction_kw - cumulative_kw)
        # system does NOT invent additional curtailment beyond eligible pool
-- RULE 11 holds even under shortfall
        transition_state(INSUFFICIENT_FLEXIBILITY)   # Part L state machine,
added per external review

    RETURN selected, cumulative_kw
```

Required reduction > available flexibility: logged explicitly as an `insufficient_flexibility` event (Part S: dr_events table gets a flag), never papered over — this is a real, expected outcome given Phase 1's finding that participation will likely be well under 100%, and the dashboard (Part T) should surface it honestly rather than hide it.

O. Fairness

`fairness_penalty_i` (used in Part N's score) rises with: recent interruption count (last 24h), cumulative interruption duration (last 7 days), and falls with days since last participation — implemented as a simple normalized rolling score, not a separate ML model. A household/appliance that was curtailed in the last cycle is pushed toward the back of the ranking next cycle, all else equal, so the same few appliances aren't repeatedly targeted. Rotation is a natural side-effect of this penalty term, not a separate mechanism.

P. Rebound Management

Problem: all curtailed appliances restoring at the exact same instant creates a new, sharper spike right after the event (Phase 1 §20).

Design: staggered restart — selected appliances are assigned a restart offset (e.g. spread evenly across a 5-15 minute window, C. system assumption to tune in Phase 3/implementation) rather than a single restore command to all. Priority-based restart order: appliances with the longest max_interruption_minutes or highest household-stated urgency restore first within the window. A randomized jitter (a few seconds per appliance) is added on top of the scheduled offset to avoid synchronized restarts even within the same offset bucket.

Measurement: rebound_peak_kw = max(feeder_load) observed in the 15 minutes immediately following event end, compared against the pre-event baseline peak — logged as its own metric (Part X) so a large rebound isn't hidden inside an otherwise-good peak-reduction number.

Q. Failure and Safety Model

Failure	Safe behavior
LoRa communication failure (simulated)	Household Controller defaults to fail-open: appliance resumes/continues normal operation, never stuck curtailed
Gateway failure (simulated)	Same fail-open default propagates to all households behind that gateway
Backend failure	No new commands issued; already-dispatched curtailments respect their own pre-set max_interruption_minutes and self-restore locally even without a backend restore command (a local timeout, not a backend dependency)
Forecast failure/error (model exception)	Stress Detection falls back to "no forecast available" state — treated as Normal (no DR triggered) rather than guessing, since a false Critical trigger is worse than a missed one at this stage
Incorrect forecast (wrong but not erroring)	Not detectable in real time by the system itself; surfaced only in post-hoc evaluation (forecast MAE/RMSE, Part X) — logged, not silently corrected
Appliance unavailable (already off)	Excluded from flexibility calculation (Part M), not an error condition
User opt-out mid-event	Immediate removal from the active event, appliance restores at next cycle
User override	Same as opt-out — immediate, unconditional

Failure	Safe behavior
Multiple simultaneous events (e.g. two feeders sharing infrastructure)	Out of scope for Phase 2/MVP — each feeder simulated independently; flagged as FUTURE WORK
Insufficient flexibility	Logged explicitly (Part N), never fabricated
Power failure at household	Appliance state simply becomes unreadable/off; treated the same as "appliance unavailable"
Sensor failure (future physical hardware)	Out of Phase 2 software scope; physical-layer failure handling is a Level 2/3 hardware concern (Part AG)
Duplicate commands	Commands are idempotent by design (re-sending "curtail" to an already-curtailed appliance is a no-op)
Delayed commands	Timestamped and checked against event validity window on arrival; a command arriving after its event has ended is discarded, not executed late

Governing principle (RULE 2 realized), wording corrected per external review: "fail-open" is ambiguous once real relay hardware is involved (relay normally-open/normally-closed contact configuration determines actual physical behavior, and "open" doesn't map cleanly to "appliance runs normally" in every wiring). The precise requirement is: **fail-safe normal-operation behavior** — loss of communication, gateway, or backend availability must never leave an appliance unintentionally curtailed; the local Household Controller restores or maintains normal operation using a local timeout, independent of backend availability. Every failure branch above resolves to "appliance behaves as if FeederFlex didn't exist," never to "appliance stuck in an unsafe or user-unintended state." (This replaces all prior uses of "fail-open" in this document with this more precise phrasing.)

R. Communication Architecture

```
Household Controller <--LoRa (simulated in Phase 2)--> Gateway <--uplink (simulated)--> Backend
```

Message types and example payloads (all JSON for the simulation layer; a real LoRa payload would be a compact binary encoding of the same fields, out of Phase 2 scope):

```
// 1. Appliance status (household -> backend, periodic heartbeat + on change)
{"type": "status", "household_id": "H-014", "appliance_id": "A-051", "state":
"on", "power_w": 620, "timestamp": "..."}

// 2. Flexibility status (household -> backend)
{"type": "flexibility", "household_id": "H-014", "usable_kw": 0.8,
"timestamp": "..."}

// 3. DR command (backend -> household)
{"type": "dr_command", "appliance_id": "A-051", "action": "curtail",
"duration_minutes": 20, "event_id": "E-2201"}

// 4. Acknowledgement (household -> backend)
{"type": "ack", "appliance_id": "A-051", "event_id": "E-2201", "status":
"applied"}

// 5. User opt-out (household -> backend, via USSD layer)
{"type": "opt_out", "household_id": "H-014", "scope": "event", "event_id":
"E-2201"}

// 6. Override (household -> backend)
{"type": "override", "appliance_id": "A-051", "action": "resume_now"}

// 7. Heartbeat (household -> backend, liveness only)
{"type": "heartbeat", "household_id": "H-014", "timestamp": "..."}
```

Simulation vs. future physical communication: in Phase 2, all of the above are function calls / in-memory events inside the simulator, not real radio packets. If Level 2/3 physical LoRa hardware is built later, the Household Controller's simulated message-send function is the exact point where it gets replaced with a real LoRa transmit call — the message schema above is designed to be reused unchanged.

S. Database Design (PostgreSQL / Supabase-compatible)

Table	Key columns	Types	PK/FK	Purpose
users	id, phone_number, created_at	UUID, text (hashed/masked), timestamptz	PK id	Login/identity for household users (simulated phone numbers only, no real PII per NFR-008)
households	id, user_id, feeder_id, created_at	UUID, FK, FK, timestamptz	PK id, FK user_id→users, FK feeder_id→feeders	One household per user in this scope
feeders	id, name, capacity_kw, safety_margin, household_count	UUID, text, float, float, int	PK id	Simulation-configured feeder (Part L parameters live here)
appliances	id, household_id, appliance_type, rated_power_w, criticality, interruptible, shiftable, current_state	UUID, FK, enum, float, enum, bool, bool, enum	PK id, FK household_id→households	The Part F data model
appliance_preferences	id, appliance_id, user_permission, max_interruption_minutes, cooldown_minutes, preferred_window_start, preferred_window_end	UUID, FK, enum, int, int, time, time	PK id, FK appliance_id→appliances	User-set flexibility rules
household_preferences	id, household_id, opted_in, paused_until	UUID, FK, bool, timestamptz nullable	PK id, FK household_id→households	Household-level participation state
feeder_readings	id, feeder_id, timestamp, actual_load_kw	UUID, FK, timestamptz, float	PK id, FK feeder_id→feeders	Ground-truth simulated feeder load per timestep
household_loads	id, household_id, timestamp, load_kw	UUID, FK, timestamptz, float	PK id, FK household_id→households	Per-household simulated load, source for feeder_readings aggregation
appliance_loads	id, appliance_id, timestamp, power_w, state	UUID, FK, timestamptz, float, enum	PK id, FK appliance_id→appliances	Per-appliance simulated load, source for household_loads
forecast_results	id, feeder_id, timestamp, horizon_minutes, predicted_load_kw, model_version	UUID, FK, timestamptz, int, float, text	PK id, FK feeder_id→feeders	Part K outputs, joined against feeder_readings for MAE/RMSE
flexibility_snapshots	id, feeder_id, timestamp, theoretical_kw, available_kw, usable_kw	UUID, FK, timestamptz, float, float, float	PK id, FK feeder_id→feeders	Part M's three-tier output over time

Table	Key columns	Types	PK/FK	Purpose
dr_events	id, feeder_id, start_time, end_time, required_reduction_kw, achieved_reduction_kw, insufficient_flexibility, rebound_peak_kw	UUID, FK, timestamptz, timestamptz nullable, float, float, bool, float nullable	PK id, FK feeder_id→feeders	One row per Part J event cycle
dr_actions	id, dr_event_id, appliance_id, action, dispatched_at, restored_at, was_overridden	UUID, FK, FK, enum(curtail/restore), timestamptz, timestamptz nullable, bool	PK id, FK dr_event_id→dr_events, FK appliance_id→appliances	Part N's individual dispatch/restore log, feeds Part X metrics
user_overrides	id, household_id, appliance_id nullable, dr_event_id nullable, override_type, timestamp	UUID, FK, FK nullable, FK nullable, enum, timestamptz	PK id, FKs as noted	Part G/Q override tracking
ussd_sessions	id, household_id, session_log, timestamp	UUID, FK, jsonb, timestamptz	PK id, FK household_id→households	Part H simulated session records
incentives	id, household_id, dr_event_id, credited_amount, credited_at	UUID, FK, FK, numeric, timestamptz	PK id, FK household_id→households, FK dr_event_id→dr_events	Incentive/status data shown in USSD flow
audit_logs	id, actor_type, actor_id, action, details, timestamp	UUID, enum, UUID, text, jsonb, timestamptz	PK id	NFR-007 observability requirement

ER diagram (Mermaid)

```
erDiagram
    users ||--|| households : has
    feeders ||--o{ households : serves
    households ||--o{ appliances : owns
    appliances ||--|| appliance_preferences : configured_by
    households ||--|| household_preferences : configured_by
    feeders ||--o{ feeder_readings : logs
    households ||--o{ household_loads : logs
    appliances ||--o{ appliance_loads : logs
    feeders ||--o{ forecast_results : predicted_for
    feeders ||--o{ flexibility_snapshots : estimated_for
    feeders ||--o{ dr_events : triggers
    dr_events ||--o{ dr_actions : contains
    appliances ||--o{ dr_actions : target_of
    households ||--o{ user_overrides : issues
    households ||--o{ ussd_sessions : conducts
    households ||--o{ incentives : earns
    dr_events ||--o{ incentives : generates
```

T. API Design (REST, FastAPI-shaped)

Method	Path	Purpose	Input	Output	Facing
POST	/api/auth/register	Create user + household	phone_number	user_id, household_id	User
GET	/api/households/{id}	Household detail	-	household + preferences	User
POST	/api/households/{id}/opt-in	Opt in/out of DR	opted_in bool	confirmation	User
POST	/api/households/{id}/pause	Temporary opt-out	paused_until	confirmation	User
POST	/api/appliances	Register an appliance	household_id, type, rated_power_w	appliance_id	User
PUT	/api/appliances/{id}/preferences	Set flexibility permission	user_permission, max_interruption_minutes, window	confirmation	User
GET	/api/appliances/{id}	Appliance detail + current state	-	appliance record	User
GET	/api/feeders/{id}/status	Current + forecast load, flexibility, active events	-	Part Y-shaped dashboard payload	Operator
GET	/api/feeders/{id}/forecast	Forecast series	horizon param	forecast_results rows	Operator, internal
GET	/api/feeders/{id}/flexibility	Current flexibility snapshot	-	Part M data structure	Operator, internal

Method	Path	Purpose	Input	Output	Facing
POST	/api/feeders/{id}/simulate	Run a simulation cycle/experiment	simulation config (Part V)	run_id, summary metrics	Internal/demo
GET	/api/dr-events/{id}	Event detail	-	dr_event + dr_actions	User, operator
POST	/api/dr-events/{id}/override	Household override on an active event	appliance_id, action	confirmation	User
GET	/api/households/{id}/history	Past events, interruptions	date range	list of dr_actions	User
GET	/api/households/{id}/incentives	Incentive/status	-	incentives rows, running total	User
POST	/api/ussd/session	Simulated USSD session step	session_id, keypress	menu state, response text	User (simulated)
POST	/api/internal/forecast/run	Trigger a forecast cycle	feeder_id	forecast_results row	Internal ML service
POST	/api/internal/dr/run-cycle	Trigger one DR decision cycle (Part J/N)	feeder_id	dr_event summary	Internal service
GET	/api/analytics/experiments/{run_id}	Retrieve experiment results (Part W/X)	-	metrics table	Internal/demo

Authorization: User-facing endpoints scoped to the requesting household's own household_id (household isolation, Part AA security). Operator-facing endpoints are read-only in Phase 2 (no write path exists for the DISCOM/operator actor, per Part C). Internal endpoints are not exposed outside the backend's own service boundary in the eventual deployment (called directly by the simulation orchestrator, not over public HTTP, in a real build).

U. Frontend Design

Note: the household's real interface is USSD (Part H), simulated. These screens are the **demo/judge-facing** and **operator-facing** surfaces, plus a USSD simulator screen so judges can see that interaction without a real phone.

Screen	Purpose	Components	Data shown	API calls	User actions	Error states
Landing/Log in	Entry point, choose role (household demo / operator view)	Role selector	-	-	Pick a role	-
Household Dashboard	Household's own view (for demo purposes, not the real interface)	Status card, appliance list	Opted-in state, appliance summary	GET /households/{id}	Navigate to appliance mgmt	Household not found
Appliance Management	Register/edit appliances	Form, list	Registered appliances	POST/GET /appliances	Add, edit appliance	Invalid power rating
Appliance Preferences	Set flexibility permission per appliance	Permission selector (never/limited/conditional), duration input	Current preferences	PUT /appliances/{id}/preferences	Save preferences	Out-of-range duration

Screen	Purpose	Components	Data shown	API calls	User actions	Error states
DR Event View	Show an active/past event affecting this household	Event card, countdown	Event status, affected appliances	GET /dr-events/{id}	Override/resume now	No active event
Opt-Out / Override	Pause or unregister	Confirmation dialog	Current opt-in state	POST /opt-in, /pause	Confirm pause/unregister	-
Incentive/History	Show accumulated credit and past events	Table, running total	incentives, history rows	GET /incentives, /history	Filter by date	Empty history
USSD Simulator	Let a judge experience the actual household interface	Simulated phone keypad + screen	Live menu state	POST /ussd/session	Press keys, navigate menu	Session timeout
DISCOM Dashboard	Operator's feeder view (Part Y schema)	Feeder map/list, load chart, flexibility gauge, event log	feeder status, forecast, flexibility range, active events	GET /feeders/{id}/status	Select feeder, view event detail	No live data (explicitly labeled simulated)
Feeder Details	Drill into one feeder	Charts: baseline vs controlled load	forecast_results, feeder_readings	GET /feeders/{id}/forecast	Change time range	-
Simulation Dashboard	Configure and run an experiment (Part V/W)	Config form, run button	Simulation config	POST /feeders/{id}/simulate	Set households/participation/severity, run	Invalid config
Analytics/Results	Show Part X metrics for a run	Metric cards, comparison charts	Peak reduction %, MAE/RMSE, rebound, etc.	GET /analytics/experiments/{run_id}	Compare runs	Run not found

Stack for these screens: React + TypeScript + Vite, Tailwind for styling, Recharts for the load/flexibility charts — evaluated and confirmed reasonable for hackathon timeframe in Part Z.

V. Simulation Engine

Configuration object:

```
{
  "run_id": "auto-generated",
  "households": 100,
  "resolution_minutes": 15,
  "duration_hours": 24,
  "participation_rate": 0.7,
  "feeder_capacity_kw": 150.0,
  "safety_margin": 0.12,
  "dr_window": {"start": "17:00", "end": "20:00"},
  "shortfall_severity": "moderate",
  "appliance_flexibility_profile": "default",
  "forecast_model": "xgboost",
  "random_seed": 42
}
```

Inputs: household count, household/appliance load profiles (drawn from the real datasets per Part F's source legend), feeder capacity, participation rate, forecast model choice, shortfall severity, DR window.

Outputs: baseline load series, controlled load series, all Part X metrics, dr_events/dr_actions logs, per-run summary JSON for the Analytics screen.

Determinism: a fixed random_seed makes household sampling, participation assignment, and any randomized restart jitter reproducible — required for NFR-001 and for judges to be able to re-run a demo and get the same result.

W. Experiment Design

Experiment	Question	Independent variable	Dependent variables	Controls
A. Baseline vs FeederFlex	Does the DR mechanism reduce peak/shortfall risk at all?	DR on/off	Peak reduction %, shortfall-risk reduction	Same household set, same shortfall event
B. Participation rate sweep	How sensitive is the result to opt-in rate?	participation_rate at 30/50/70/90%	Peak reduction %, insufficient-flexibility rate	Same feeder, same event
C. Feeder size sweep	Does the mechanism hold at different scales?	households at 50/100/150/200	Peak reduction %, compute latency (NFR-004)	Same participation rate
D. Shortfall severity sweep	Does the controller degrade gracefully under a bigger gap?	shortfall_severity (mild/moderate/severe)	Achieved vs required reduction, insufficient-flexibility rate	Same feeder/participation

Experiment	Question	Independent variable	Dependent variables	Controls
E. Appliance flexibility assumptions	How much does the appliance mix matter?	appliance_flexibility_profile (pump/cooler-heavy vs geyser-heavy, per Phase 1 §14)	Peak reduction %, comfort-violation rate	Same feeder size/participation
F. Forecast model comparison	Does XGBoost/LightGBM actually beat the linear baseline on this data?	forecast_model (linear vs XGBoost vs LightGBM)	MAE, RMSE	Same train/test split
G. Fairness / interruption distribution	Is curtailment burden spread reasonably, or concentrated on a few households?	Number of DR cycles run	Gini-style spread of cumulative_interruptions across households	Fixed participation rate
H. Rebound analysis	How much does staggered restart actually help vs simultaneous restart?	restart_strategy (staggered vs simultaneous)	rebound_peak_kw	Same event, same selected appliances

Per RULE 16, no numerical results are asserted here — these are the experiments to run, not their outcomes. Each experiment's plots/tables should follow directly from Part X's metric definitions, and interpretation should explicitly separate "what the simulation shows" from "what that implies about the real world," given the synthetic-feeder caveat (Part Y).

X. Evaluation Metrics

#	Metric	Formula	Why it matters
1	Peak reduction %	(baseline_peak_kw − controlled_peak_kw) / baseline_peak_kw × 100	Headline number, benchmark against Karnataka (~2%) and POWBAL (8.5-15%) from Phase 1 §3/§19
2	Overload duration reduction	baseline_minutes_over_threshold − controlled_minutes_over_threshold	Captures duration of stress, not just peak magnitude
3	Energy shifted	Σ (baseline_load_t − controlled_load_t) for t in DR window, where positive	Total kWh curtailed/shifted during the window

#	Metric	Formula	Why it matters
4	Available flexibility	usable_after_constraints_kw at event trigger time (Part M)	Reported as the min/expected/max range, never a single guaranteed number
5	Flexibility utilization %	achieved_reduction_kw / usable_after_constraints_kw × 100	How much of the available pool the controller actually used
6	Forecast MAE	mean(|actual − predicted|) over test set	Standard forecast accuracy
7	Forecast RMSE	sqrt(mean((actual − predicted)^2)) over test set	Penalizes large forecast misses more than MAE
8	Shortfall-risk reduction	(baseline critical-state minutes − controlled critical-state minutes) / baseline critical-state minutes × 100	Ties directly to Part L's state machine
9	Comfort/constraint violation rate	count(dr_actions where max_interruption_minutes exceeded or minimum_runtime violated) / total dr_actions	Should be zero by construction (RULE 11) — this metric exists to prove the constraint logic actually holds, not to tolerate violations
10	Rebound peak	max(feeder_load) in the 15 min after event end, vs. pre-event baseline peak (Part P)	Catches a hidden cost of aggressive curtailment
11	Household interruption fairness	standard deviation (or Gini coefficient) of cumulative_interruptions across opted-in households	Low spread = fair distribution (Part O)
12	Number of controlled appliances	count(distinct appliance_id in dr_actions) per event	Basic scale/reach indicator

#	Metric	Formula	Why it matters
13	DR event success rate	count(dr_events where achieved_reduction_kw >= required_reduction_kw) / total dr_events	Directly surfaces the insufficient_flexibility rate from Part N

Every metric here is computable from the tables defined in Part S — no metric requires data this architecture doesn't already collect.

Y. Real vs Simulated Components

This is the section to have ready when a judge asks "which parts are actually real?"

Component	Status	Source	Implementation
Household/appliance historical load shapes	**REAL**	iAWE, Kaggle Indian smart-meter datasets, Batra et al. ownership data (subject to Phase 3 re-verification, RULE 3)	Direct dataset ingestion
Weather / solar radiation	**REAL**	Open-Meteo, live and historical, confirmed free/available in Phase 1 §28	Live API calls
National/state grid demand context	**REAL**	Grid-India (POSOCO), confirmed live in Phase 1 §28	API/scrape, used for contextual framing only, not feeder-level input
The feeder itself (50-200 specific households, their identities, their exact load)	**SIMULATED**	Constructed by aggregating real household profiles with a diversity factor	Synthetic feeder builder (Part J, F)
Household opt-in/participation	**SIMULATED**	No real households are enrolled	participation_rate parameter, Part V

Component	Status	Source	Implementation
DR event outcomes	**SIMULATED**	No real appliances are curtailed	Simulation engine, Part V/N
Forecasting model	**REAL** (as software) trained on **REAL** historical data, applied to a **SIMULATED** feeder	XGBoost/LightGBM per Part K	Genuinely trained and evaluated, just not on a real feeder
DR controller / optimization logic	**REAL** (as software), operating on simulated inputs	Part N pseudocode, fully implementable and testable	Genuinely running code, not a mockup
USSD/missed-call layer	**MOCKED**	No telecom aggregator integration	State-machine simulator, Part H
LoRa communication	**MOCKED** in software; **OPTIONAL PHYSICAL** demo possible	No real radio traffic unless Level 2/3 hardware is built	Function calls standing in for messages, Part R
DISCOM SCADA/DMS interface	**MOCKED**, designed for compatibility	No DISCOM access exists	Dashboard schema shaped for IEC 60870-5-104, read-only, no live connection
Physical relay/ESP32/LoRa hardware	**OPTIONAL PHYSICAL**	Not required for MVP (Part AG)	2-3 breadboard nodes if time allows
Real-world participation rate, cost comparisons, feeder-level forecast accuracy	**FUTURE INTEGRATION** — explicitly not claimed	N/A	Would require a real DISCOM pilot (RULE 20)

One-sentence answer for a judge: "The data and the algorithms are real — the forecasting model is genuinely trained on real Indian household data, and the DR controller genuinely runs. The feeder, the households on it, and their participation are simulated, because no real low-income feeder dataset exists publicly, and no live DISCOM/telecom integration was pursued in this phase."

Z. Technology Stack — Evaluated and Frozen

Layer	Candidate	Evaluated against	Verdict
Frontend	React + TypeScript + Vite + Tailwind + Recharts	Hackathon time, needs charts + forms only, no exotic UI	**Keep.** Recharts over ECharts specifically for faster setup; ECharts only worth it if more complex chart types are needed later
Backend	Python + FastAPI	Needs to serve the same language as the ML stack (no cross-language serialization overhead), fast to stand up	**Keep.** Single-language backend+ML is the right call for a small team under time pressure
ML	pandas, NumPy, scikit-learn, XGBoost, LightGBM (if justified per Part K), joblib	Matches Part K's model recommendations exactly	**Keep as specified**
Database	PostgreSQL / Supabase	Schema in Part S is fully relational, no need for a document store; Supabase gives auth+DB+hosting in one for hackathon speed	**Keep**, with SQLite as an even-lower-friction fallback for local dev/demo if Supabase setup costs more time than it saves

Layer	Candidate	Evaluated against	Verdict
Real-time	WebSocket	Only needed if the dashboard should show live-updating simulation runs	**Conditional keep** — only implement if Phase 13 (integration) has time left; polling the API every few seconds is an acceptable fallback and simpler to build
Communication	LoRa (concept only), USSD (simulated)	Matches Part R/H exactly	**Keep, simulated only, per scope in Part B**
Deployment	Vercel (frontend) + Render/Railway (backend) + Supabase (DB)	Free/cheap tiers, minimal DevOps for a hackathon	**Keep**

Overall verdict: the originally proposed stack survives evaluation essentially as-is — it was already well-matched to the project's actual complexity (Phase 1 explicitly warned against over-engineering with RL/deep learning/complex solvers, and this stack doesn't introduce any of that). The only two conditional items are WebSocket (nice-to-have, not required) and Supabase-vs-SQLite (pick based on how much setup friction the team wants to absorb).

AA. Project Directory Structure

```
feederflex/
  frontend/                # React + TS + Vite app (Part U screens)
    src/
      pages/                # one folder per screen in Part U
      components/
      api/                  # typed client for Part T endpoints
  backend/                 # FastAPI app
    app/
      api/                  # route handlers, one module per Part T resource
group
      models/               # ORM models matching Part S schema
```

```
      services/
        forecasting/          # Part K
        stress_detection/     # Part L
        flexibility/          # Part M
        dr_controller/        # Part N, O, P
        communication/        # Part R message simulation
        ussd/                 # Part H state machine
      core/                   # config, auth, db session
  ml/
    data_ingestion/           # pulls Open-Meteo, Grid-India, loads
Kaggle/iAWE files
    feature_engineering/
    training/                  # Part K training scripts, model versioning
    models/                    # serialized joblib artifacts
    notebooks/                  # exploration, not production code
  simulation/
    feeder_builder/           # Part F/J synthetic feeder construction
    engine/                   # Part V simulation orchestration
    experiments/              # Part W experiment scripts
  data/
    raw/                       # untouched downloaded datasets
    processed/                 # cleaned/resampled data
  docs/
    phase1_research_audit/     # this conversation's first doc
    phase2_system_spec/        # this document
  tests/
    backend/
    ml/
    simulation/
  hardware/                    # OPTIONAL, Level 2/3 only
    firmware/                  # ESP32 relay+LoRa code, if built
```

Responsibility split: `simulation/` never imports from `backend/` directly — it's designed to run standalone (for Part W experiments) and be called *by* the backend's `/simulate` endpoint (Part T), not the other way around, so the research/experiment work isn't blocked on the API being finished.

AB. Development Dependency Graph

Stage	Prerequisite	Input	Output	Definition of done
1. Database schema	None	Part S design	Migrations applied, empty tables exist	Schema matches Part S, can insert/query every table
2. Dataset processing	Stage 1 (optional, can run standalone first)	Raw iAWE/Kaggle/Open-Meteo files (Phase 3-verified per Part AC)	Cleaned, resampled 15-min household + weather data	Processed files pass basic sanity checks (no negative power, no missing timestamps beyond a documented tolerance)
3. Synthetic feeder builder	Stage 2	Processed household profiles	A feeder-level load time series for N households	Feeder load equals sum of sampled households; diversity factor applied and documented
4. Baseline simulation (no DR)	Stage 3	Feeder time series	Baseline load curve, baseline peak	Reproducible given a fixed seed (NFR-001)
5. Forecast model	Stage 3	Feeder time series + weather features	Trained model, MAE/RMSE on test set	Beats the linear-regression baseline, or documents why it doesn't
6. Flexibility engine	Stage 3 (appliance model, Part F)	Appliance states + preferences	Three-tier flexibility output (Part M)	Matches the worked example structure in Part M
7. DR controller	Stages 5, 6	Forecast + flexibility + constraints	Selected appliances, dispatch log	Never selects a non-consented/critical/cooldown appliance (RULE 11) — test this explicitly
8. Evaluation	Stages 4, 7	Baseline + controlled runs	Part X metrics computed	All 13 metrics compute without error on a full run
9. Backend API	Stages 1, 3-8 wrapped as services	Part T design	Working endpoints	Each endpoint in Part T responds correctly against Stage 1's schema
10. USSD simulator	Stage 9	Part H design	Working state machine	A full menu walkthrough (join → set preference → opt out) works end to end

Stage	Prerequisite	Input	Output	Definition of done
11. Frontend	Stage 9	Part U design	Working screens	Each Part U screen renders real data from the backend, not mock JSON
12. Real-time layer (conditional)	Stage 11	WebSocket design (Part Z)	Live-updating dashboard	Only attempted if time remains after Stage 11
13. Optional hardware	None (can run in parallel with 1-12)	Part F/R message schema	2-3 working relay/LoRa nodes	Nodes report status and accept a curtail command matching Part R's schema
14. Deployment	Stages 9, 11	Part Z deployment targets	Publicly reachable demo URL	Judges can open the link and interact with it

Critical path: 1 → 2 → 3 → (5 and 6 in parallel) → 7 → 8 → 9 → 11 → 14. Stage 13 (hardware) and Stage 12 (WebSocket) are explicitly off the critical path and should not block anything else — this matches Phase 1's own prioritization (§24) of simulation over hardware.

AC. Phase 3 Data Requirements — Exact Handoff

Dataset classification added per external review — these datasets provide genuinely different kinds of information and should not be treated as interchangeable "household-level data":

- **Dataset A -- Load consumption:** household load, timestamp, power (Kaggle Indian smart-meter datasets)
- **Dataset B -- Appliance profiles:** appliance, rated power, usage/operating patterns (iAWE)
- **Dataset C -- Appliance ownership:** probability a household owns a pump/geyser/cooler/etc. (Batra et al.)
- **Dataset D -- Weather:** temperature, humidity, solar radiation (Open-Meteo)
- **Dataset E -- Renewable generation/supply (NEW, required by the Part L/K2 renewable-shortfall fix):** used to build the synthetic PV conversion model (Part K2) and, if accessible, to validate it against real regional generation-mix data (Grid-India) -- **VERIFY IN PHASE 3**, not assumed available at feeder-scenario granularity

The mandatory/optional/fallback table below is organized by Dataset A-D; Dataset E's handling is specified separately since it's synthetic-by-construction, not a direct ingestion target.

MANDATORY DATA

Dataset	Source	Why needed	Variables needed	Resolution	Geographic relevance	License/access	Preprocessing	Fallback
Indian household/appliance load data	iAWE (NILMTK ecosystem)	Real appliance-level shapes for the synthetic feeder (Part J/F)	Whole-house + appliance power	Sub-metered, high-frequency	India (single house) — **VERIFY IN PHASE 3** whether more than one household is actually in the public release	**VERIFY IN PHASE 3** — confirm current access/license terms	Resample to 15-min, map appliances to Part F's appliance_type enum	Kaggle Indian smart-meter datasets below
Indian smart-meter household data	Kaggle "Electricity smart meter data from Indian two cities" (Mathura, Bareilly)	Additional real household load series to diversify the synthetic feeder beyond one house	Household consumption time series	High-frequency smart meter	India, 2 cities	**VERIFY IN PHASE 3** — Kaggle dataset license terms and whether download still active	Resample to 15-min, align timestamps	Kaggle "Indian Household Electricity Consumption Dataset"
Weather/solar data	Open-Meteo	Forecasting features (Part K)	Temperature, cloud cover, shortwave radiation, humidity	Hourly (interpolate to 15-min)	Any Indian coordinates	Free, no key, confirmed in Phase 1 §28 — **re-confirm the 10,000 calls/day limit still applies at Phase 3 time**	Align to feeder timestamps, forward/interpolate to 15-min	OpenWeather Map (paid) if Open-Meteo access changes

OPTIONAL DATA

Dataset	Why optional	Notes
Batra et al. Indian household appliance-ownership data	Improves realism of which appliances a synthetic household "has," but the feeder can be built without it using reasonable default appliance-mix assumptions (documented as C. system assumptions, Part AD)	**VERIFY IN PHASE 3** — confirm public access to the actual dataset, not just the paper

Dataset	Why optional	Notes
NEEM-linked 13-city Indian smart-meter dataset	Would allow climate-zone diversity in the synthetic feeder	**VERIFY IN PHASE 3** — the Phase 1 audit found this referenced inside a published study, not confirmed as an independently downloadable public dataset; treat as unconfirmed until checked directly
Grid-India / Grid Status India real-time demand data	Contextual/narrative use only ("here's what the national grid looks like right now") — not a required input to any calculation in this spec	Free tier vs paid API tier — **VERIFY IN PHASE 3**

FALLBACK DATA (if mandatory sources are inaccessible at Phase 3 time)

- If iAWE access is blocked: fall back entirely to the Kaggle Indian smart-meter datasets and document the reduced appliance-level granularity.
- If no real Indian appliance-level dataset is accessible at all: construct household profiles from documented, cited **typical appliance power ratings** (e.g. published manufacturer specs for pumps/geysers/coolers) combined with a synthetic usage-pattern generator — but this must be labeled as **SIMULATED**, not REAL, in Part Y, and the Assumption Register (Part AD) must record this fallback explicitly.

DO NOT WASTE TIME COLLECTING

- Real-time feeder-level SCADA data — confirmed not publicly available in Phase 1 §18/§28; do not spend Phase 3 time searching further without a DISCOM contact.
- A live USSD/telecom aggregator sandbox — out of scope per Part B/AF; not needed for the software deliverable.
- Battery/storage cost benchmarks beyond what's already cited in Phase 1 §17 — the audit already flagged this as an open gap; a full LCOE-style analysis is not required for the hackathon prototype.

AD. Assumption Register

ID	Assumption	Why needed	Source	Type	Risk	How to validate	Phase to validate
A-01	5-8pm evening window is the correct DR target	Frames the whole trigger design	Phase 1 §3, EAC-PM national data	Research-supported	Low — well-evidenced nationally	Confirm pattern holds for the specific target city if one is chosen	Phase 3/write-up
A-02	100 households, 15-min resolution is a reasonable starting feeder	Simulation default	Phase 1 §16	Engineering	Low	Sensitivity-tested via Part W Experiment C	Implementation
A-03	feeder_capacity_kw and safety_margin values	Required for Part L's trigger to run at all	No real feeder capacity data exists	Simulation/engineering	Medium — arbitrary values could make results look better/worse than realistic	Sweep a plausible range, report sensitivity rather than one fixed number	Implementation
A-04	Appliance mix defaults (pump/cooler-heavy, geyser cautious)	Drives which appliances are eligible by default	Phase 1 §14, POWBAL device-level findings	Research-supported (partially — POWBAL was AC-heavy, not directly this segment)	Medium	Ideally validated against Batra et al. ownership data if accessible (Part AC)	Phase 3
A-05	Ranking weights (w1-w5) in Part N's scoring formula	Required for the algorithm to run	No literature gives exact weights for this context	Engineering	Medium — arbitrary weighting could produce non-intuitive selections	Manual review of selected-appliance lists against expectation; tune empirically	Implementation
A-06	Participation rate defaults (30/50/70/90% sweep)	Required for Experiment B	POWBAL's real rate was under 1%, but that was smartphone-gated (Phase 1 §3)	Simulation assumption, explicitly a sweep not a point estimate	Low, given it's presented as a sweep, not a claim	Would require a real pilot to validate a true rate	Future work, not Phase 2/3
A-07	Restart stagger window (5-15 minutes)	Required for Part P	No specific source; general engineering judgment	Engineering	Low-medium	Measure rebound_peak_kw across different window sizes	Implementation
A-08	Diversity factor for feeder aggregation	Required so synthetic feeder isn't a naive sum of coincident peaks	Standard distribution-engineering practice, not sourced to a specific Indian value here	Engineering	Medium	Compare against published diversity-factor ranges if found in Phase 3	Phase 3

AE. Risk Register

Risk	Probability	Impact	Mitigation	Validation method
Mandatory dataset (iAWE) access changes or license terms block use	Low-medium	Medium — fallback exists (Part AC)	Use Kaggle fallback datasets	Confirm access at the start of Phase 3, not the end
Synthetic feeder doesn't resemble a real low-income feeder's actual load shape	Medium	Medium — undermines the realism of results	Explicitly label all outputs as simulated (Part Y); sensitivity-test across appliance-mix assumptions (Experiment E)	Ongoing, inherent limitation, can't be fully resolved without real data
Forecast accuracy is poor on the synthetic feeder	Medium	Low-medium — still demonstrable as a methodology even if MAE is mediocre	Report MAE/RMSE honestly, compare to the linear baseline rather than claiming an absolute accuracy target	Part X, Experiment F
Participation-rate assumptions are unrealistic	High (given POWBAL's own experience)	Medium	Present as a sweep, not a single claimed number (Experiment B)	Cannot be fully validated without a real pilot

Risk	Probability	Impact	Mitigation	Validation method
DR controller has a bug that selects a non-consented appliance	Low if tested, high impact if it happens	High — violates RULE 11, a core safety/trust requirement	Explicit unit test asserting this can never happen (Part AB Stage 7 definition of done)	Automated test suite
Communication failure handling isn't actually fail-open in the implementation	Low if tested, high impact	High — safety requirement (Part Q)	Explicit failure-injection tests	Automated test suite
Rebound staggering doesn't meaningfully reduce rebound peak	Medium	Low-medium	Report Experiment H's result honestly either way	Part X metric 10
Scope creep — team attempts live DISCOM/telecom integration	Medium (ambition is natural in a hackathon)	High — burns time on something Part AF rules out	Enforce Part AF's "DO NOT BUILD" list explicitly at kickoff	Team process, not technical
Hardware demo distracts from software depth	Medium	Low-medium	Keep hardware strictly optional and off the critical path (Part AB)	Team process
Regulatory/consent framing overclaimed to judges	Low if this spec is followed	Medium — credibility risk	Follow Part B's scope boundary and Phase 1 §16's "open legal question" framing	Pitch review before submission

AF. What NOT to Build (Hackathon MVP)

Based on the architecture above, explicitly do not attempt:

- Full real DISCOM SCADA/IEC 104 integration — design for compatibility only (Part T, Y)
- Nationwide or multi-feeder grid optimization — one synthetic feeder at a time (Part B scope boundary)
- Real telecom USSD infrastructure/aggregator contract — simulated state machine only (Part H)
- Commercial-scale LoRa deployment — at most 2-3 demo nodes (Part AG Level 2/3)
- Reinforcement learning or MILP for the DR controller — constrained heuristic ranking only, per Phase 1 §11 and RULE 14, unless Phase 3/implementation finds a specific, documented reason the heuristic genuinely fails
- A complex digital twin of the actual physical distribution network (impedances, power flow analysis) — the simulation is a load-aggregation model, not a power-flow model
- Household battery/storage management — explicitly out of scope, FeederFlex is storage-free by design (Part B)
- Rooftop solar management at the household level — not the target segment (Phase 1 problem statement)
- A mobile app — the entire point is a non-smartphone interface (Part H)
- Unnecessary IoT hardware beyond the minimal optional demo nodes — no need for sensors/features not tied to a Part F/N requirement
- Live retraining/MLOps pipeline for the forecasting model — train once, serve, per Part K
- A WebSocket real-time layer, unless Stage 11 (Part AB) finishes with time to spare

AG. MVP vs Advanced vs Future

	LEVEL 1 — MVP	LEVEL 2 — Strong competition prototype	LEVEL 3 — Research/deployment-ready
Features	Synthetic feeder (Stage 3), baseline vs. linear-regression forecast, rule-based stress detection, basic DR controller (Part N without full fairness/rebound tuning), Part X core metrics (1, 6, 7, 9, 13), a minimal dashboard (even just notebook plots)	Full Part K forecasting (XGBoost/LightGBM), full DR controller with fairness + staggered restart, full USSD simulator, full React dashboard (Part U), all Part X metrics, 2-3 experiments from Part W	All Part W experiments run and written up, physical hardware demo (Part AG hardware), a real DISCOM conversation/LOI attempted, a small primary-research step on household willingness (Phase 1 §14/§25), submission-ready as an applied-research paper
Time/effort	Achievable within Track 2's ~6-week window even with a small team, per Part AB's critical path stages 1-9	Requires the full team executing in parallel across frontend/backend/ML through the full 6 weeks	Multi-month, beyond hackathon scope — stated here as the roadmap beyond the hackathon, not a hackathon deliverable
Technical complexity	Low-moderate — every component individually simple by design	Moderate — integration across layers is the main complexity, not any single algorithm	High — requires real-world validation loops this spec cannot provide
Dependencies	Stages 1-9 of Part AB only	Adds Stages 10-11, optionally 12	Adds real DISCOM/telecom partnerships not controllable by the team

Level 1 is confirmed achievable: every component it requires (schema, synthetic feeder, linear-regression forecast, rule-based trigger, basic ranking controller, core metrics) is fully specified above with no open architectural question blocking it, other than the Phase 3 dataset-access verifications in Part AC.

AH. Final Consolidated Specification — FeederFlex System Specification v1.0

This is a pointer/index, not a restatement — every item below is fully specified in its own lettered section above, so this reads as the table of contents an implementing engineer should follow in order.

1. System objective → Part A
2. Scope → Part B
3. Actors → Part C
4. Functional requirements → Part D
5. Non-functional requirements → Part E
6. Appliance model → Part F
7. User control model → Part G
8. USSD model → Part H
9. Architecture → Part I
10. Data flow → Part J
11. Forecasting → Part K
12. Stress detection → Part L
13. Flexibility estimation → Part M
14. DR optimization → Part N
15. Fairness → Part O
16. Rebound management → Part P
17. Communication → Part R (failure/safety is Part Q, listed first since safety gates communication design)
18. Database → Part S
19. APIs → Part T
20. Frontend → Part U
21. Simulation → Part V

22. Experiments → Part W
23. Metrics → Part X
24. Technology stack → Part Z
25. Directory structure → Part AA
26. Security → below, this section
27. Failure handling → Part Q
28. Real vs simulated components → Part Y
29. MVP scope → Part AG
30. Phase 3 handoff → Part AC

Security and Privacy (Part AI's numbering slot — kept practical, not overengineered per the brief's instruction)

- **Authentication:** simple token-based auth for the demo (e.g. JWT), phone-number-based identity, no real KYC needed since all households are simulated (Part Y).
- **Authorization:** household isolation — a household's API calls are scoped to its own household_id only (Part T).
- **Command authentication:** DR commands (Part R message type 3) should carry a signed/authenticated field even in simulation, so the eventual physical-hardware path (Part AG) doesn't need this added retroactively — this was a gap identified in Phase 1 §20 and is closed here at the design stage.
- **Replay protection:** dr_command messages include event_id and are checked against already-processed event_ids before acting (prevents a duplicate/replayed command from double-curtailing).
- **Secure API:** standard HTTPS in any real deployment; not a hackathon-specific concern beyond using a standard platform (Vercel/Render) that provides it by default.
- **Audit logging:** every state-changing action logged to audit_logs (Part S), satisfying NFR-007.
- **User consent:** enforced structurally, not just by policy — RULE 11 is implemented as a hard filter in Part N's pseudocode, not a guideline.
- **Data minimization:** no real PII in Phase 2 scope (Part B, NFR-008) — simulated households only.

- **Appliance-control authorization:** an appliance can only be commanded if it belongs to a household that has both opted in at the household level (household_preferences.opted_in) and permitted that specific appliance (appliance_preferences.user_permission != never) — a two-layer check, not one.

This security section is intentionally practical-scope: it covers what a hackathon-grade implementation needs to not be embarrassingly insecure, not enterprise-grade hardening.

AI. Final Architecture Review

Architecture Issues Found

1. **Contradiction risk:** Part G says automated events "proceed unless overridden" (matching POWBAL), but Part G's step 8 also lists "Accept/decline if applicable" as if some events require active consent per-event. These two models aren't fully reconciled.
2. **Unsupported assumption:** the diversity factor used in synthetic feeder aggregation (Part AA/AD A-08) has no cited Indian-specific value — it's currently a placeholder concept, not a number.
3. **Missing component:** there is no explicit handling for a household that opts in but has zero eligible appliances at a given moment (e.g., everything already off) during a DR cycle — Part M handles this per-appliance, but the household-level UX/messaging for "you had nothing to contribute this time" isn't designed.
4. **Overengineering risk:** the three-tier flexibility model (Part M) plus the five-term scoring formula (Part N) plus fairness (Part O) plus rebound (Part P) is a lot of interacting parameters (Part AD lists 8 assumption-heavy values) for a hackathon team to tune well in the available time.
5. **Security gap (minor):** Part AI's command-authentication note says commands "should carry a signed field" but doesn't specify a concrete signing scheme — left abstract.
6. **Unrealistic hardware assumption check:** Part F's appliance model assumes rated_power_w is knowable per appliance; for real low-income households this may require user estimation (a smartphone-free household can't easily look up a wattage label mid-USSD-call) — Part F lists this as B. user configuration, which may be optimistic about data quality.
7. **Unrealistic DISCOM assumption check:** none found beyond what Phase 1 already flagged (Part Y correctly scopes this as mocked) — this part passes review.

8. **Research-paper weakness:** the experiment design (Part W) is solid, but Part AH's security section is listed as item 26 out of order relative to the brief's original AA-AJ lettering, which was intentionally resolved by treating AH as an index rather than a literal restatement — flagged here for transparency, not left silent.
9. **Hackathon demo weakness:** the USSD simulator (Part H, U) is the most "invisible" part of the demo — it's easy for a live demo to under-sell this component compared to the flashier dashboard, even though it's the differentiator.

Recommended Resolution

1. Resolve by simplifying Part G to one model: **automated-by-default with an immediate override option**, matching POWBAL's evidenced design exactly (Phase 1 §3) — drop the per-event accept/decline branch from the primary flow, keep it only as a documented alternative if the team later wants to A/B it.
2. Treat the diversity factor as an explicit Part AD assumption (already done) and assign it a placeholder value (e.g. 0.7, a common range in distribution-engineering literature generally) with a clear "not India-specific, needs Phase 3 citation or engineering justification" label — do not present it as sourced.
3. Add an explicit "no contribution this cycle" status to the household_preferences/dashboard messaging — a small addition to Part H's menu ("Current DR Event" shows "No action needed from you" as a valid state) and Part U's Household Dashboard.
4. For Level 1/MVP (Part AG), explicitly recommend implementing Parts M/N with fairness (Part O) and rebound (Part P) staggering as fixed simple defaults rather than tuned parameters — save tuning for Level 2, so Level 1 isn't blocked on getting 8 assumption values "right."
5. Leave the signing scheme unspecified at Phase 2 (implementation detail, e.g. HMAC with a shared household key, is a Phase 3/implementation decision, not an architectural one) — mark as **DATA-DEPENDENT DECISION — VERIFY IN PHASE 3/implementation**.
6. Add a fallback in Part F: if rated_power_w is unknown, the system uses a C. system-assumption default per appliance_type (e.g. a typical pump/geyser/cooler wattage range), documented in Part AD, rather than requiring the household to supply an exact figure.
7. No change needed.
8. No change needed beyond this note — the AH section's numbering deviation is intentional and explained there.

9. Recommend the demo script (outside this spec's scope, a Track 1/Track 2 delivery detail) explicitly walk the USSD simulator before the dashboard, not after, so judges see the differentiator first.

AJ. Exact Phase 3 Handoff

Phase 3's job is narrow and data-focused, per Part AC: verify access/license/resolution/columns for iAWE, the two Kaggle datasets, and Open-Meteo's current terms; confirm or rule out the Batra et al. and NEEM-linked datasets; and resolve the two DATA-DEPENDENT DECISIONs flagged in Part AI (diversity factor sourcing, command-signing scheme). Nothing else in this spec is blocked on Phase 3 — implementation (Part AB) can start on Stage 1 (database schema) immediately, in parallel with Phase 3's dataset verification.

PHASE 2 COMPLETION CHECKLIST

[x] Scope frozen (Part B) [x] Actors frozen (Part C) [x] Appliance model frozen (Part F) [x] User-control model frozen (Part G) [x] Architecture frozen (Part I) [x] Data flow frozen (Part J) [x] Forecasting design frozen (Part K) [x] Flexibility model frozen (Part M) [x] DR algorithm frozen (Part N) [x] Database frozen (Part S) [x] API design frozen (Part T) [x] Frontend screens frozen (Part U) [x] Simulation design frozen (Part V) [x] Metrics frozen (Part X) [x] Technology stack frozen (Part Z) [x] Directory structure frozen (Part AA) [x] Phase 3 data requirements frozen (Part AC, this section) [x] Assumptions documented (Part AD) [x] Risks documented (Part AE) [x] MVP scope frozen (Part AG) [x] Architecture reviewed (Part AI) — **2 issues resolved inline, 1 explicitly deferred to Phase 3/implementation as a DATA-DEPENDENT DECISION (command-signing scheme), 1 deferred as an engineering-not-architectural decision (diversity factor exact value)**

Not yet frozen, by design: the exact numeric values for tunable parameters (Part AD's assumption register) — these are meant to be set during implementation/Phase 3, not guessed here, per RULE 16's prohibition on inventing numerical results.

AK. External Review Response — v1.0 to v1.1 Changelog

An external (ChatGPT-generated) review of this spec raised 20 numbered points. Each was independently checked against Phase 1's evidence and this document's own content before acting on it -- not applied automatically.

Applied as MUST-fix (7): renewable-supply/shortfall modeling was genuinely absent and is the most important structural gap closed in this revision (Parts I, J, K2, L); the interruption-duration-vs-count bug in Part N's pseudocode was real and is fixed; the user-override/opt-out contradiction in Part G was real and is fixed with an explicit action/effect table; minimum-runtime had no backing field and now does (Part F); "fail-open" has been replaced with precise "fail-safe normal-operation" language throughout Part Q; curtailment-vs-shifting wording now matches what the MVP algorithm actually implements (Part F); renewable-aware stress detection replaced the demand-only trigger (Part L).

Applied as SHOULD-fix (8): 11 kV/DT feeder framing (Part B); dropped the "every household has >=1 flexible appliance" assumption (Part B); "low-income" labeling caveat added (Part B); Dataset A-E separation, including the new required Dataset E (Part AC); multi-day training-data requirement (Part K); forecast confidence made a required empirical-residual output, not optional (Part K); AC's conditional tier simplified for MVP (already Never/Limited-only in this spec's original Part F, confirmed adequate); scoring formula refined -- criticality removed as a redundant score term since it was already a hard filter, urgency defined explicitly, and an explicit minimum-necessary-intervention objective added (Part N).

Corrected, not simply applied: point #16's proposed formula still double-counted criticality as both a hard filter and a score term; this revision removes it from scoring rather than keeping it in both places, since the review's own stated reasoning ("put criticality/consent in hard constraints, not merely scoring") argues for removal, not retention.

Rejected as factually incorrect: point #20, that Parts S-AJ are missing. The critique was run against a PDF export that stopped at Part R. The live document has always had Parts S through AJ complete (database schema, APIs, frontend, simulation engine, experiments, metrics, technology stack, directory structure, dependency graph, Phase 3 handoff, assumption register, risk register, MVP tiers, final specification, architecture review, and completion checklist) -- this was a stale-export artifact of how the review was generated, not a real gap in the specification.

Not yet independently verified in this revision (carried forward as open items, not silently assumed true): the specific claim that NFMS monitors feeders at 11 kV (Part B item 8); whether a real regional renewable-generation-by-fuel-type dataset is usable at feeder-scenario granularity (Part K2, Part AC Dataset E) -- both marked **VERIFY IN PHASE 3** rather than treated as settled.